MONGODB

PROGRAMMING LABORATORY MANUAL

Practical Programs • MongoDB Shell • MongoDB Node.js Driver

Programming-focused compilation of the supplied MongoDB documents

1. Laboratory Folder Structure

```
MongoDB-Lab/
│
├── 01_MongoDB_Shell/
│   ├── 01_create_database.js
│   ├── 02_insert_documents.js
│   ├── 03_find_documents.js
│   ├── 04_queries_operators.js
│   ├── 05_array_queries.js
│   ├── 06_projection_sort_limit.js
│   ├── 07_delete_documents.js
│   ├── 08_update_documents.js
│   └── 09_schema_validation.js
│
├── 02_Node_MongoDB/
│   ├── package.json
│   ├── package-lock.json
│   ├── db-connection.js
│   ├── index.js
│   └── node_modules/
│
└── 03_Mini_Project_Placements/
    ├── package.json
    ├── db-connection.js
    ├── index.js
    └── README.md
```

Recommended execution order:

Order	Program	Main skill
1	Create/use database and collection	Database setup
2	Insert documents	insertOne(), insertMany()
3	Read documents	find(), findOne()
4	Comparison and logical queries	$gt, $lt, $gte, $lte, $or, $and
5	Nested/array queries	Dot operator, $elemMatch, $in
6	Projection, sorting and limit	Field selection and ordering
7	Delete documents	deleteOne(), deleteMany()
8	Update documents	$set, $inc, upsert, $push, $pull
9	Schema validation	$jsonSchema
10	Node.js MongoDB driver	Express + MongoDB connection
11	Placements mini-project	Integrated CRUD/query task

Program 1: Create/Select Database and Collection

Task: Open mongosh, select moviesdb, verify the database and collections, and inspect the movies collection.

```
show dbs
use moviesdb
db

db.createCollection("movies")
show collections
db.movies.find()
```

Expected result: The shell switches to moviesdb and displays the available collections/documents.

Program 2: Insert One Document

Task: Insert a movie document into the movies collection.

```
db.movies.insertOne({
  name: "2012",
  genres: ["fiction"],
  producer: "jake maa",
  rating: 9,
  reviews: [
    { name: "dannis", message: "Nice" },
    { name: "David", message: "Very good movie" }
  ]
})
```

Expected result: MongoDB returns acknowledged: true and an insertedId.

Program 3: Insert Multiple Documents

Task: Insert multiple movie documents in one operation.

```
db.movies.insertMany([
  {
    name: "Titanic",
    genres: ["romantic"],
    release_date: "01-01-1990",
    producer: "James Cameron",
    reviews: [
      { name: "Tim", message: "Good movie" },
      { name: "Jone", message: "Great movie" }
    ],
    rating: 5
  },
  {
    name: "Pulp fiction",
    genres: ["fiction", "science"],
    release_date: "01-01-2023",
    producer: "Romeo",
    reviews: [
      { name: "Ryan", message: "Good movie" },
      { name: "Victor", message: "Good movie" }
```

```
    ],
    rating: 7
  }
])
```

Expected result: MongoDB returns acknowledged: true and the insertedIds object.

Program 4: Find Documents

Task: Display all documents, retrieve one document, and retrieve a document by ObjectId.

```
db.movies.find()

db.movies.findOne()

db.movies.findOne({
  _id: ObjectId("655c60a1d0192fcba43c7787")
})
```

Expected result: Matching documents are displayed in the shell.

Program 5: Comparison and Logical Queries

Task: Perform queries using comparison and logical operators.

```
// Greater than
db.movies.find({ rating: { $gt: 8 } })

// Less than
db.movies.find({ rating: { $lt: 8 } })

// Greater than or equal to
db.movies.find({ rating: { $gte: 8 } })

// Less than or equal to
db.movies.find({ rating: { $lte: 8 } })

// OR
db.movies.find({
  $or: [
    { name: "2012" },
    { name: "abcde" }
  ]
})

// AND
db.movies.find({
  $and: [
    { name: "2012" },
    { rating: 9 }
  ]
})
```

Expected result: Only documents satisfying the specified conditions are returned.

Program 6: Queries on Nested Documents and Arrays

Task: Search values inside an array of subdocuments and array fields.

```
// Match a complete review object
db.movies.find({
  reviews: { name: "Tim", message: "Good movie" }
})

// Dot operator
db.movies.find({
  "reviews.name": "imran"
})

db.movies.find({
  "reviews.message": "Good movie"
})

// Array element match
db.movies.find({
  reviews: {
    $elemMatch: { name: "Ryan" }
  }
})

// Match an item in an array
db.movies.find({
  genres: { $in: ["horror"] }
})
```

Expected result: Documents containing the requested nested/array values are returned.

Program 7: Projection, Sorting and Limiting

Task: Display selected fields, sort documents, and restrict the number of returned documents.

```
// Projection
db.movies.find(
  { genres: "horror" },
  { name: 1, genres: 1, rating: 1 }
)

// Exclude a field
db.movies.find(
  { genres: "horror" },
  { reviews: 0 }
)

// Ascending sort
db.movies.find({}).sort({ name: 1 })

// Descending sort
db.movies.find({}).sort({ name: -1 })

// Limit result
db.movies.find({}).sort({ name: 1 }).limit(2)
```

Expected result: The result contains only the selected fields and/or the requested number/order of documents.

Program 8: Delete Documents

Task: Delete one matching document, multiple matching documents, and all documents in a collection.

```
// Delete one document
db.movies.deleteOne({
  _id: ObjectId("655cac72d76a5a5a1df10e53")
})

// Delete all matching documents
db.movies.deleteMany({
  genres: "fiction"
})

// Delete all documents
db.movies.deleteMany({})
```

Expected result: MongoDB returns acknowledged: true with the deletedCount.

Program 9: Update Documents

Task: Modify documents using the update operators demonstrated in the source material.

```
// $set
db.movies.updateOne(
  { _id: ObjectId("655cb05fd76a5a5a1df10e63") },
  { $set: { name: "New The Matrix!" } }
)

// $inc
db.movies.updateOne(
  { _id: ObjectId("655cb05fd76a5a5a1df10e63") },
  { $inc: { rating: 1 } }
)

db.movies.updateOne(
  { _id: ObjectId("655cb05fd76a5a5a1df10e63") },
  { $inc: { rating: -1 } }
)

// upsert
db.movies.updateOne(
  { name: "Lagaan" },
  { $set: { name: "Lagaan", producer: "Amir khan", rating: 9 } },
  { upsert: true }
)

// $push
db.movies.updateOne(
  { _id: ObjectId("655cb05fd76a5a5a1df10e63") },
  { $push: { genres: "science" } }
)
```

```
// $pull
db.movies.updateOne(
  { _id: ObjectId("655cb05fd76a5a5a1df10e63") },
  { $pull: { genres: "science" } }
)

// updateMany
db.movies.updateMany(
  { genres: "fiction" },
  { $set: { status: "updated" } }
)
```

Expected result: The matching documents are modified and MongoDB reports matchedCount/modifiedCount/upsertedCount.

Program 10: JSON Schema Validation

Task: Create a students collection with JSON Schema validation for name, age and course.

```
db.createCollection("students", {
  validator: {
    $jsonSchema: {
      title: "Student Object Validation",
      required: ["name", "age", "course"],
      properties: {
        name: {
          bsonType: "string",
          description: "Name must be a string and is required"
        },
        age: {
          bsonType: "int",
          minimum: 5,
          maximum: 20,
          description: "Age must be an integer between 5 and 20"
        },
        course: {
          bsonType: "string",
          enum: ["BCA", "BTech", "BSc"],
          description: "Course must be one of BCA, BTech, BSc"
        }
      }
    }
  }
})

db.students.insertOne({
  name: "Amir",
  age: 20,
  course: "BTech"
})

db.students.find()
```

Expected result: Valid documents are accepted; documents violating the defined schema are rejected.

2. Node.js + MongoDB Driver

Execute this section after completing the MongoDB shell programs.

Program 11: Create Node.js Project

Create the project directory and install the packages used in the supplied Node.js example.

```
mkdir NODE-MONGO-LAB
cd NODE-MONGO-LAB

npm init -y
npm i express mongodb
```

Program 12: package.json

```
{
  "name": "mongo-yt",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "start": "node --watch index.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongodb": "^6.3.0"
  }
}
```

Program 13: db-connection.js

```
const { MongoClient } = require("mongodb");

const getMongoDBConnection = async () => {
  try {
    const dbUrl = "mongodb://127.0.0.1:27017/moviesdb";
    const client = await MongoClient.connect(dbUrl);
    return client.db();
  } catch (err) {
    throw err;
  }
};

module.exports = {
  getMongoDBConnection
};
```

Program 14: index.js – Connect Express to MongoDB

```
const express = require("express");
const app = express();
const { getMongoDBConnection } = require("./db-connection");
```

```
let db;

async function init() {
  try {
    db = await getMongoDBConnection();
    console.log("MongoDB connected...");

    app.listen(8080, () => {
      console.log("Server is running on port:8080");
    });
  } catch (err) {
    console.log("Error", err);
  }
}

init();
```

Program 15: Fetch Movies Using Express

```
// Fetch all the movies
app.get("/movies", async (req, res) => {
  try {
    const cursor = await db.collection("movies").find({});
    const movies = await cursor.toArray();

    res.status(200).json(movies);
  } catch (err) {
    res.status(500).json("Internal server error");
  }
});
```

Run:

```
npm start
```

Test in a browser/API client:

```
http://localhost:8080/movies
```

3. Mini Project – Placement Database

Create a collection named placements and perform the following programming tasks.

```
use placementdb

db.placements.insertMany([
  {
    studentId: 101,
    studentName: "Student A",
    companyName: "Company A",
    packageOffered: 9,
    year: 2026
  },
  {
    studentId: 102,
    studentName: "Student B",
    companyName: "Company B",
    packageOffered: 7,
    year: 2026
  },
  {
    studentId: 103,
    studentName: "Student C",
    companyName: "Company A",
    packageOffered: 12,
    year: 2026
  }
])
```

Required Queries

```
// 1. Students placed with packages above  8 LPA
db.placements.find({
  packageOffered: { $gt: 8 }
})

// 2. Highest package offered
db.placements.find().sort({
  packageOffered: -1
}).limit(1)

// 3. Count students placed company-wise
db.placements.aggregate([
  {
    $group: {
      _id: "$companyName",
      count: { $sum: 1 }
    }
  }
])

// 4. Display top 5 packages
db.placements.find().sort({
  packageOffered: -1
}).limit(5)
```

4. Practical Submission Checklist

- Create the folders in the sequence shown in Section 1.
- Save each MongoDB shell program with the indicated filename.
- Execute each program in order and retain the command/output screenshots.
- Create the Node.js project only after completing the MongoDB shell programs.
- Keep db-connection.js and index.js in the same Node.js project folder.
- Run the Express application using npm start and test /movies.
- Complete the placements mini-project using the required fields and queries.

Source Coverage

This manual consolidates the programming material, command sequence, MongoDB query examples, JSON Schema validation examples, Node.js MongoDB-driver project structure, and placement task from the two supplied documents. Theory/explanatory slides have been intentionally omitted.